

MartPoint WhatsApp Sales Flow — Zero API Cost

Prepared for: MartPoint Product Team

Date: 2026-09-15

Goal: Let customers sell/place orders through WhatsApp without paying Meta WhatsApp Business API conversation fees. Inventory is still updated automatically when the merchant confirms the order.

1. Why this design?

- The **Meta WhatsApp Business Platform** charges per user conversation. At high volume this becomes expensive for small African retailers.
- **wa.me** links are completely free. They open the customer's WhatsApp app with a pre-filled message.
- MartPoint already has the pieces needed:
 - `db_online_orders` table
 - `Storefront_model::adjustStock()` and `restoreStock()` for inventory
 - `Pos_model::update_items_quantity()` for full stock recalculation
 - Sendchamp / email for notifications

This flow uses **only free WhatsApp features** for the customer and the merchant. MartPoint handles the inventory, order record, and notifications.

2. The flow (step by step)

Customer side

1. Customer sees the product — on the MartPoint storefront, an Instagram/TikTok link, a QR code, or a shared catalog.
2. Taps **“Order on WhatsApp”**.
3. MartPoint instantly:
 - Checks live stock,
 - Creates a **draft order** in `db_online_orders`,
 - **Reserves** the stock for 15 minutes,
 - Generates a signed confirmation token.
4. Customer is redirected to a `wa.me/{store_number}?text=...` link.
5. The message is already typed in their WhatsApp. Customer taps **Send**.

Merchant side

1. Merchant receives the WhatsApp message. It contains:
 - Customer name & phone
 - Product / quantity / total
 - A one-tap **Confirm Order** link
2. Merchant taps the confirm link (open in mobile browser).
3. MartPoint validates the signed token and, if still valid:
 - Finalizes the order,
 - Deducts the reserved stock from db_items and db_warehouseitems,
 - Marks order as confirmed.
4. Customer receives an SMS / email receipt. Merchant can also reply manually on WhatsApp for a personal touch.

What happens if the merchant does not confirm?

- After 15 minutes the draft expires.
 - MartPoint automatically cancels the draft and **restores** the reserved stock.
 - No manual work needed.
-

3. URL examples

Customer message link (wa.me)

`https://wa.me/2348036028069?text=Hi%2C%20I%20want%20to%20order%3A%0A%F0%9F%9B%92%20Rice%205kg%20`

The customer only has to press **Send**.

Merchant confirm link

`https://martpoint.ng/omni/wa/confirm/abc123XYZ`

- Signed with an HMAC secret tied to the store.
 - Single-use.
 - Expires with the draft order.
-

4. Data model additions

Reuse db_online_orders

Add the following columns:

Column	Type	Purpose
source_channel	VARCHAR(20)	whatsapp / web / pos
channel_user_id	VARCHAR(50)	

Column	Type	Purpose
		Customer phone number from WhatsApp
confirmation_token	VARCHAR(128)	Signed token for the merchant confirm link
token_expires_at	DATETIME	When the draft auto-expires
stock_reserved	TINYINT(1)	Whether stock is currently reserved

New: db_item_reservations (recommended)

Column	Type	Purpose
id	INT PK	—
item_id	INT	Product
warehouse_id	INT	Warehouse/branch
qty	DECIMAL	Reserved quantity
reservation_token	VARCHAR(128)	Ties back to the draft order
expires_at	DATETIME	Auto-expiry
status	VARCHAR(20)	reserved / converted / released

A reservation table prevents overselling across POS, storefront, and WhatsApp at the same time.

5. Inventory rules

Stage	Action	Code path to call
Draft created	Reserve stock, do not deduct	db_item_reservations insert
Merchant confirms	Convert reservation to real deduction	Storefront_model::adjustStock() + Pos_model::update_items_quantity()
Draft expires / order cancelled	Release reservation	Storefront_model::restoreStock()
Refund	Restore stock	Storefront_model::restoreStock()

6. Cost table

Item	Cost
Customer clicks wa.me link	Free
Customer sends WhatsApp message	Free (uses customer data)

Item	Cost
Merchant confirms order	Free
SMS / email confirmation to customer	Existing provider cost only (Sendchamp / email)
WhatsApp Business API	Not used — \$0

7. Implementation phases

Phase 1 — MVP (recommended first sprint)

1. Add **“Order on WhatsApp”** button to product and storefront mobile/desktop views.
2. Create POST /omni/wa/draft endpoint:
 - Validate stock,
 - Create draft db_online_orders record,
 - Reserve stock for 15 minutes,
 - Generate signed wa.me URL.
3. Create GET /omni/wa/confirm/{token} endpoint:
 - Validate token and expiry,
 - Convert draft to confirmed order,
 - Deduct inventory,
 - Send SMS/email receipt.
4. Background job to auto-release expired drafts (cron every minute or on-demand).

Phase 2 — Merchant quick actions

- Add a **“Confirm / Cancel”** quick-actions screen in the MartPoint mobile dashboard for incoming WhatsApp drafts.
- Allow merchant to add a quick-reply message in WhatsApp Business app (manual) for fast confirmation.

Phase 3 — Optional API (only if volume justifies it)

- If later you want fully automated WhatsApp replies, payment buttons, or delivery updates, add the **Meta WhatsApp Cloud API**.
- Keep Phase 1 + 2 as the zero-cost fallback.

8. Security & risk notes

- **Token abuse:** confirm links are HMAC-signed, single-use, and expire with the draft.
- **Overselling:** reservation table + expiry + re-validation at confirm time prevents it.
- **Merchant friction:** one-tap confirm; no need to type.

- **Customer follow-up:** use SMS/email so you never need a WhatsApp API for outbound.

9. Files likely to touch

Layer	File	Change
Controller	application/ controllers/Omni.php (new) or Storefront.php	Draft and confirm endpoints
Model	application/models/ Storefront_model.php	Reservation, confirm, and expiry logic
Model	application/models/ Pos_model.php	Full warehouse stock recalc after confirm
View	Storefront product / mobile product views	Add Order on WhatsApp button
View	application/views/ storefront/ order_received.php	Optional “share on WhatsApp” thank-you
Cron	application/ controllers/Cron.php or new Omni_cron.php	Release expired reservations

10. How people know you have WhatsApp ordering

Discovery matters. The best flow is useless if no one clicks it. Built-in discovery points:

Customers

1. **Product page CTA:** Every product detail page now has a green “**Order via WhatsApp**” button below “**Add to Cart**”.
2. **Catalogue cards:** Product/service cards show a **WhatsApp Enquire** link on the storefront.
3. **QR codes / flyers:** Each product can be shared as `https://{store}.martpoint.ng/store/{slug}/product/{id}`. Customers scan and tap **Order via WhatsApp**.
4. **Social bios:** Link the storefront in Instagram / TikTok / Snapchat bios. Customers land on the product page and click the WhatsApp CTA.
5. **Order thank-you page:** After a successful web order, existing customers can “Share on WhatsApp” to refer friends.

Merchants

1. **Dashboard banner:** The **Online Store Dashboard** now shows a green banner: *“Sell on WhatsApp — zero API cost”* plus the count of pending WhatsApp orders.
2. **Orders list filter:** Orders placed via WhatsApp appear in Online Store > Orders with Payment Method = whatsapp.
3. **Settings toggle:** Online Store > Settings already has **Allow WhatsApp Orders**. Keep this ON.
4. **Quick-launch marketing copy:** Use the templates in this doc to announce the channel on your social pages:
 - **Instagram/TikTok caption:** *“Order straight from our WhatsApp! Tap ‘Order via WhatsApp’ on any product — no app, no API fees, just checkout and chat.”*
 - **WhatsApp status:** *“New on our store: click ‘Order via WhatsApp’ on any product, and we’ll confirm your order in one tap.”*
 - **Store sign / flyer:** *“Shop on WhatsApp — zero extra cost. Visit {store_url} or message {whatsapp_number}.”*

Public announcement checklist

- ☐ Update Instagram/TikTok bio link to the storefront.
 - ☐ Post one product with the **“Order via WhatsApp”** CTA highlighted.
 - ☐ Add a WhatsApp status or broadcast with the store link.
 - ☐ Print a QR code poster with <https://wa.me/{number}> or the storefront URL.
 - ☐ Mention it in your next email/SMS campaign via Sendchamp.
-

11. Decision needed

The first phase is now implemented: wa.me draft + confirm endpoints, the storefront **“Order on WhatsApp”** button, and the merchant dashboard callout. The next step is to monitor early orders, then decide whether to add automatic stock reservation and the optional WhatsApp Business API for fully automated replies.